CHULETA — Desarrollo Web en Entorno Cliente · RA1–RA3

Teoría · cuestionario RA3 · cómo resolver ejercicios · respuestas correctas marcadas

RA1 · Modelo cliente-servidor

Servidor: hardware/software + CGI, APIs, PHP/ASP, servlets Java, módulos Apache (`mod_access` acceso, `mod_actions` CGI/HTTP, `mod_auth` autenticación).

Cliente: navegador + HTML, CSS, JavaScript, ActiveX.

Red: protocolos y conectividad.

Características de JavaScript

- Script **interpretado** (sentencia a sentencia, sin compilar).
- **Débilmente tipado y dinámico**: sin declarar tipo; puede cambiar en ejecución.
- **Orientado a objetos** y a **prototipos**.
- **Imperativo y estructurado** (secuencia, selección, iteración).
- **No sustituye al servidor**: no escribe ficheros en servidor; sí los solicita. La web debe funcionar sin JS.
- Código **visible**; la ofuscación es reversible (no protege).

4 capas del desarrollo web

- Comportamiento → **JavaScript**
- Presentación → **CSS**
- Estructura → **DOM/HTML**
- Contenido → texto, imágenes, vídeo

4 tecnologías base: HTML (estructura), CSS (presentación), JavaScript (comportamiento), AJAX (integración asíncrona).

Arquitectura de un navegador

- Interfaz de **usuario**
- **Motor de navegación** (interfaz de alto nivel)
- **Motor de renderizado** (representación visual)
- **Comunicaciones** (HTTP, FTP, HTTPS)
- **Intérprete de JavaScript**
- **Parser XML** (árbol en memoria)
- **Visualización**
- **Persistencia de datos**

Criterios de clasificación

Plataforma/SO · características · personalización/accesibilidad · soporte de estándares · licencia (libre/propietario).

Firefox abierto y multiplataforma · **Chrome** compilado de componentes abiertos · **Safari** Apple · **Opera** propietario con componentes abiertos (escritorio Windows/Mac/Linux; móvil iOS/Android).

Lenguajes y tecnologías

- **HTML**: marcas, interpretado, hipervínculos.
- **XML**: describe/transfiere datos, no los muestra.
- **XHTML**: HTML adaptado a XML.
- **DHTML**: HTML + JS + CSS + DOM.
- **CSS**: separa formato de estructura.
- **AJAX**: técnicas asíncronas; hoy JSON más que XML.
- **Flash/ActionScript**; **applets Java** (Chrome y Edge ya no los soportan).

Consola y vocabulario

Chrome `Ctrl+Shift+I` · Firefox `Ctrl+Shift+K`.

- **Lenguaje**: Angular→TypeScript, Symfony/Laravel→PHP, Spring→Java.
- **Librería**: dependencia para una funcionalidad (jQuery).
- **Framework**: caja de herramientas del proyecto.
- **VanillaJS**: JavaScript puro.
- **Frontend** = cliente · **Backend** = servidor.

Integrar JS en HTML

1. `<script>...</script>` en el documento.
2. Fichero externo `<script src="...">` (sin límite de ficheros).
3. En atributos (`onclick="..."`): peor mantenimiento.

Ventajas del externo: carga más rápida (caché), separa estructura/comportamiento, depuración y mantenimiento, modularidad, seguridad.

RA2 · Identificadores y variables

Letras, números, `$` y `_`. Primer carácter **no** número ni `@`. `@` no vale en **ninguna** posición. `10_nombre` y `mi@nombre` → incorrectos.

	Ámbito	Reasignable	Hoisting
`var`	global/función (no bloque)	sí	sí
`let`	global/función/bloque	sí	no (TDZ)
`const`	global/función/bloque	no	no

`const` no se reasigna, pero objetos/arrays son **mutables**. `var` en bucles anidados comparte variable y se pisa; `let` crea una por bloque.

```
console.log(x); // undefined
var x = 1;
console.log(b); // ReferenceError (TDZ)
let b = 1;
```

typeof

```
typeof undefined // "undefined"
typeof null      // "object"  <-- trampa
typeof "texto"   // "string"
typeof 15        // "number"
typeof true      // "boolean"
```

undefined: no existe o no se asignó. **null**: existe, asignado sin valor. `undefined == null` true; `===` false. `NaN === NaN` false; usar `isNaN()`.

== vs ===: usar `===`. Con `==`: mismo tipo igual que estricto; string vs número convierte el string; booleano vs otro convierte a número. `''==0`, `'5'==5`, `false==0` → true.

Strings

- Comillas simples, dobles o backticks.
- `length` cuenta caracteres: `"Mi\n".length` → **3**.
- **Inmutables**: no cambia un carácter; se reasigna la variable.
- `s[-1]` → `undefined`; `s.at(-1)` → último.
- Escapes: `\n \r \t \v \' \" \\ \xdd \u0008`.
- Template: `` `1 + 2 = ${1 + 2}.` `` → evalúa la expresión.

Números y entrada

`Number()`, `parseInt(str, base 2–36)`, `parseFloat()`. `parseInt('2n9c',36)`→`123456`; `parseInt('0xff',16)`→`255`.

`alert()`, `confirm()` (true/false), `prompt()` (string/null). Comentarios `//` y `/* */`.

RA3 · Date y Math

`getDate()` 1–31 · `getMonth()` **0–11** (+1) · `getFullYear()` · `getDay()` 0=domingo · `toLocaleTimeString()` · `setInterval(fn,1000)/clearInterval`.

`Math.PI`, `E`, `exp(x)`, `pow(b,e)`, `sqrt(x)`, `abs(x)`, `random()` [0,1), `max(...arr)`, `min(...arr)`.

```
Math.floor(-1.6) // -2 (abajo)
Math.ceil(-1.6)  // -1 (arriba)
Math.round(-1.6) // -2 (más cercano)
Math.trunc(-1.6) // -1 (quita decimales)
```

Number

`MAX_VALUE`, `MIN_VALUE`, `MAX_SAFE_INTEGER`=9007199254740991, `NaN`, `±Infinity`.

`toFixed(n)` → **string** con n decimales (rellena ceros); `+` unario lo hace number. `toPrecision(n)` → string con n cifras significativas. **BigInt** `10n`: no mezclar con Number ni Math; usar `**`.

String — métodos

`length`, `toString()`, `concat()`, `toUpperCase()`, `toLowerCase()`, `indexOf()` (-1), `lastIndexOf()`, `substring(i,f)` (fin excluido; negativos→0; intercambia), `slice(i,f)` (acepta negativos), `split(sep)` (sin arg coma; `""` letras), `replace()` (1ª), `replaceAll()`, `charAt()`, `charCodeAt()`, `includes()`, `startsWith()`, `endsWith()`, `trim()`, `repeat()`, `padStart()`, `padEnd()`.

Array

`length` modificable: `arr.length=0` **vacía**; subirla después **no recupera**.

`push/pop` final · `unshift/shift` inicio · `indexOf`, `includes`, `join(sep)` · `slice` (copia) · `splice` (muta) · `sort` (muta; texto por defecto; numérico `(a,b)=>a-b`) · `reverse` (muta) · `at(-1)` último; `arr[-1]` undefined.

Referencia: `let b=a` copia la referencia. Copiar: `[...a]`, `structuredClone()`.

Objetos nativos

- **Boolean**: `new Boolean(false)` es objeto **truthy**.
- **Object**: `keys`, `values`, `entries`, `assign(destino,fuente)`, `freeze`.
- **JSON**: `stringify(obj)` ↔ `parse(texto)`.
- **Promise**: `then`, `catch`, `finally`, `all`, `race`.

Map y Set

```
const m = new Map();
m.set("nombre","Ana"); m.set("edad",25);
m.set("nombre","Luis"); // sobrescribe
m.size;        // 2
m.get("nombre"); // "Luis"
m.get("pais");   // undefined
m.has("edad");  // true
m.delete("edad"); m.clear();
```

Map: claves únicas, de cualquier tipo. **Set**: valores únicos; `add`, `delete` (true/false), `has`, `size`, `clear`. Deduplicar: `[...new Set(a)]`.

Control de flujo

`if/else if/else`, `switch` con `break`/`default`, `for`, `while`, `do-while`. `for-in` → índices/claves; `for-of` → valores (no objetos literales). `break`, `continue`, `return`.

BOM

Jerarquía: `window` → `document`, `navigator`, `screen`, `history`, `location`.

- **window**: `innerWidth/Height`, `alert/confirm/prompt`, `open/close`, `setTimeout/clearTimeout`, `setInterval/clearInterval`, `scrollTo`, `localStorage/sessionStorage`.
- **navigator**: `appCodeName`, `appName`, `appVersion`, `cookieEnabled`, `platform`, `userAgent`, `plugins`.
- **screen**: `height`, `width`, `availHeight`, `availWidth`, `colorDepth`, `pixelDepth`.
- **history**: `length`, `back()`, `forward()`, `go(n)`.
- **location**: `href`, `protocol`, `host`, `hostname`, `port`, `pathname`, `search`, `hash`; `reload()`, `replace()`, `assign()`.
- **document**: `getElementById`, `getElementsByName`, `getElementsByTagName`, `write`, `writeln`, `cookie`, `title`, `URL`, `referrer`; `forms[]`, `images[]`, `links[]`, `anchors[]`.

Almacenamiento web

`localStorage` **no expira** · `sessionStorage` se borra al cerrar sesión. ≥**5 MB**, nunca se envía al servidor, más seguro que cookies. `setItem`, `getItem`, `removeItem`, `clear`. Soporte: `typeof(Storage) !== "undefined"`.

Funciones

- `return` corta la ejecución; nº de argumentos puede diferir.
- `arguments`: parecido a array; **no existe en arrow**.
- Valores por defecto solo si falta o es `undefined`.
- Requerido: default que lanza `throw new Error(...)`.
- **Arrow** `(a,b)=>a+b`: anónima, sin `arguments`, `this` léxico.
- **Spread ...**: concatenar/copiar arrays, string→array, fusionar objetos (gana el último), pasar argumentos.

Objetos propios

1) Literal `{marca:"LG"}` · 2) `new Object()` · 3) función constructora + `new`. `this` es **dinámico** (depende de la llamada). JS usa **prototipos**, no clases reales.

Programación funcional

```
arr.every(x => cond)      // true si TODOS
arr.some(x => cond)       // true si AL MENOS UNO
arr.filter(x => cond)     // array con los que cumplen
arr.find(x => cond)       // primer elemento o undefined
arr.findIndex(x => cond)  // índice o -1
arr.map(x => nuevo)       // array transformado
arr.reduce((acc,x)=>acc+x, 0) // acumula
```

`every/filter/find/findIndex/map/reduce/some` **no mutan**. Módulos: `export/import`; HTML `<script type="module">`.

CUESTIONARIO RA3 · respuestas

1. Strings inmutables y template literal

```
let str = `Mi\n`;
str.length;             // 3
let s = 'Hola';
s = 'h' + s[1] + s[2] + s[3];
s, s[-1];               // hola undefined
`1 + 2 = ${1 + 2}.`;    // 1 + 2 = 3.
```

Correcta: 3 / hola undefined / 1 + 2 = 3.

`\n` es **un** carácter. El `.` final es texto literal. `s[-1]` no da el último; usar `at(-1)`.

2. sort y map

```
let a = [1, 2, 15];
a.sort();                  // [1, 15, 2]
a.sort((x,y) => y - x);    // [15, 2, 1]
["Bilbo","Gandalf","Nazgul"]
  .map(i => i.length);     // [5, 7, 6]
```

Correcta: 1,15,2 / 15,2,1 / 5,7,6

`sort()` sin comparador ordena como **texto** (`"15" < "2"`) y muta. `map` devuelve array nuevo.

3. Math y toFixed

```
Math.floor(-1.6) // -2
Math.ceil(-1.6)  // -1
Math.round(-1.6) // -2
Math.trunc(-1.6) // -1
price.toFixed(5)         // "12.34000"
typeof price.toFixed(5)  // "string"
typeof +price.toFixed(5) // "number"
```

Correcta: -2 -1 -2 -1 / 12.34000 string number

`toFixed` devuelve **string**; el `+` unario lo convierte a number.

4. Referencia de arrays y length

```
let fruits = ["Apples","Pear","Orange"];
let cart = fruits;
cart.push("Banana");
fruits.length;     // 4
fruits.length = 0; // vacía
cart.length;       // 0
cart.at(-1);       // undefined
cart[-1];          // undefined
fruits.length = 5;
fruits[3];         // undefined
fruits.length;     // 5
```

Correcta: 4 / 0 undefined undefined / undefined 5

`cart` y `fruits` son el mismo array. Truncar es **irreversible**.

5. Map

```
mapa.set("nombre","Ana");
mapa.set("edad",25);
mapa.set("nombre","Luis");
mapa.size;          // 2
mapa.get("nombre"); // "Luis"
mapa.get("pais");   // undefined
```

Correcta: 2 / Luis / undefined

Claves únicas: `set` sobrescribe. Clave inexistente → `undefined`, no `null` ni error.

6. Literales numéricos y parseInt

```
0b101               // 5
1_000               // 1000
(255).toString(16)  // "ff"
parseInt('2n9c', 36) // 123456
parseInt('0xff', 16) // 255
```

Correcta: 5 1000 / ff / 123456 255

`0b` binario; `_` separador visual; base 36 usa 0–9 y A–Z.

Trampas del examen

1. `typeof null` → `"object"`.
2. `NaN !== NaN`; usar `isNaN`.
3. `sort()` sin comparador ordena como texto.
4. `Math.round(-1.6)` → `-2`; `trunc(-1.6)` → `-1`.
5. `toFixed()` devuelve **string**.
6. `arr.length = 0` vacía y no se recupera.
7. `let b = a` copia la **referencia**.
8. `Map.get` inexistente → `undefined`.
9. `s[-1]` → `undefined`; usar `at(-1)`.
10. `substring` no acepta negativos; `slice` sí.
11. `for-of` no recorre objetos literales; `for-in` da claves.
12. `new Boolean(false)` es truthy.
13. `const` impide reasignar, no mutar.
14. `var` no respeta bloque; `let` sí.
15. `localStorage` no expira; `sessionStorage` sí.

Métodos que más caen

Quiero…	Método
último carácter	`s.at(-1)`
posición o -1	`indexOf()`
trozo sin mutar	`slice()`
trozo mutando	`splice()`
texto → array	`split()`
array → texto	`join()`
ordenar números	`sort((a,b)=>a-b)`
transformar array	`map()`
filtrar array	`filter()`
sumar array	`reduce()`
buscar uno	`find()`
comprobar todos/alguno	`every()` / `some()`
quitar duplicados	`[...new Set(a)]`
decimales como texto	`toFixed(n)`
objeto → JSON	`JSON.stringify()`
JSON → objeto	`JSON.parse()`

Salidas rápidas

```
`Mi\n`.length        // 3
"hola"[-1]           // undefined
[1,2,15].sort()      // [1,15,2]
[1,2,15].sort((a,b)=>a-b) // [1,2,15]
[1,2,3].map(x=>x*2)  // [2,4,6]
[1,2,3].filter(x=>x>1) // [2,3]
[1,2,3].reduce((a,b)=>a+b,0) // 6
[1,2,3].find(x=>x>1) // 2
[1,2,3].findIndex(x=>x>1) // 1
[1,2,3].every(x=>x>0) // true
[1,2,3].some(x=>x>2)  // true
[...new Set([1,1,2])] // [1,2]
(255).toString(16)    // "ff"
(12.34).toFixed(1)    // "12.3"
parseInt("42px")      // 42
Number("12.5")        // 12.5
+"12.34000"           // 12.34
typeof null           // "object"
NaN === NaN           // false
```

CÓMO RESOLVER LOS EJERCICIOS · patrones del repo cliente2

Formularios

- Leer: `document.getElementById("id").value`; checkbox/radio → `.checked`; select → `.value`.
- Validar con array de errores: `let errores=[]` → reglas → `if(errores.length) alert(...) else ...`.
- Evitar envío: `onsubmit="return false"` o `form.addEventListener("submit", e => e.preventDefault())`.
- Habilitar botón: `<button disabled>` y luego `btn.disabled=false` o `removeAttribute("disabled")`.
- Esperar DOM: `document.addEventListener("DOMContentLoaded", ...)`.

Validaciones típicas

```
// DNI: 8 números + letra (módulo 23)
const letras = "TRWAGMYFPDXBNJZSQVHLCKE";
const num = dni.slice(0, -1), letra = dni.at(-1).toUpperCase();
valido = /^\d{8}$/.test(num) && letra === letras[num % 23];

// IBAN ESXX
/^ES\d{2}$/.test(iban)
// CCC: 20 dígitos
/^\d{20}$/.test(ccc)
// Teléfono fijo: 8 o 9 + 8 dígitos
/^[89]\d{8}$/.test(tel)
// Matrícula: 1er dígito impar + 4 dígitos + P/S
/^[13579]\d{4}[PS]$/.test(matricula)
// Email
/^[^@\s]+@[^@\s]+\.[^@\s]+$/.test(correo)
```

DOM

- Seleccionar: `getElementById`, `querySelector`, `querySelectorAll`.
- Crear e insertar: `document.createElement("tr")`, `appendChild()`, `innerHTML`, `textContent`.
- Estilos/clases: `el.style.display="none"`, `el.classList.add("activo")`.
- Tabla dinámica: crear `table` → `tr` → `td` → rellenar → `appendChild`.

Date y Math

```
// Fecha dd/mm/aaaa
f.getDate() + "/" + (f.getMonth()+1) + "/" + f.getFullYear()
// Nombre del mes / día con arrays
meses[f.getMonth()]; dias[f.getDay()]
// Entero aleatorio entre min y max (incluidos)
Math.floor(Math.random()*(max-min+1)) + min
// Sin repetidos
while (arr.length < n) {
  const x = Math.floor(Math.random()*max)+1;
  if (!arr.includes(x)) arr.push(x);
}
// Primo
for (let i=2; i<=Math.sqrt(n); i++) if (n%i===0) return false;
// Reloj
setInterval(pintar, 1000);
```

Arrays

- Recorrer: `for (const x of arr)`, `forEach`, `map`.
- Suma/media: `reduce((a,b)=>a+b,0)`; media = suma/`length`.
- Duplicados: `[...new Set(arr)]`.
- Orden numérico: `sort((a,b)=>a-b)`.
- Buscar: `find` (elemento/undefined), `findIndex` (-1), `filter`.
- `arr[5]="x"` crea huecos y sube `length`; `length=0` vacía.

Objetos, clases y módulos

```
class Persona {
  nombre; edad;
  constructor(nombre, edad) { this.nombre = nombre; this.edad = edad;
}
  saludar() { return `Hola ${this.nombre}`; }
}
const p = new Persona("Ana", 25);
```

- Matriz con array de arrays: `plantas[i][j]`; añadir planta: `plantas.push([])`.
- Módulos: `export { Clase }` / `import { Clase } from "./Clase.js"`; HTML `<script type="module">`.
- `this` dentro de métodos apunta al objeto; arrow no tiene `this` propio.
- Spread en llamadas: `objeto.metodo(...tiendas)`; parámetros rest `...props`.

Ejercicio tipo examen (CentroComercial)

- Clase con propiedades: `nombre`, `direccion` (array), `plantas` (null al inicio).
- `agregarPlantasYLocales(n,m)`: doble `for`; cada planta es un array de locales vacíos.
- `asignarTiendas(...tiendas)`: comprobar `plantas[planta]`, crear si falta, asignar `plantas[planta][local]=nombre`.
- `imprimeLocales()`: doble `for` con `if (plantas[i][j] !== undefined)`.
- Respetar nombres de métodos y salidas por consola del enunciado.

Ejercicio tipo examen (registro + tabla + localStorage)

- Botones `type="button"` con `onclick`: no envían formulario, no hace falta `preventDefault()`.
- Validar y salir pronto: `if (...) { msg.innerHTML="..."; return; }`.
- Edad: resta de años y corrige si no ha cumplido: `if (mes<0 || (mes===0 && hoy.getDate()<nac.getDate())) edad--;`.
- Nota: `parseFloat()` + `toFixed(2)` (string). ID 1000–9999: `Math.floor(Math.random()*9000)+1000`.
- Tabla clásica: `tbody.insertRow(-1)` → `fila.insertCell(i)` → `celda.innerHTML = ...`.
- Persistir: `localStorage.setItem("clave", valor)`; al cargar `window.onload` + `Number(...)`; borrar con `removeItem`.
- Tras insertar: `form.reset()`. Vaciar todo: `tbody.innerHTML = ""` + variables a 0 + `removeItem`.

Checklist de examen

1. No modificar el HTML si el enunciado lo prohíbe.
2. Leer entradas, salidas y formato exacto (mensajes, decimales).
3. Probar casos límite: vacío, cero, negativos, fuera de rango.
4. Usar los `id/name` exactos del HTML.
5. Probar en consola antes de entregar; comprobar errores en rojo.
6. Comentar poco y claro; no dejar `console.log` de prueba si no lo piden.